Royal University of

JIGME NAMGYEL ENGINEERING COLLEGE

SMART HOUSING

Rental House Finder System

Dechen Pem
(05250170)

Abstract

Smart Housing is a Python desktop application designed to make rental-property searching and listing management easier. It uses Tkinter/ttk for the graphical interface and SQLite for persistent storage. Users can browse rental listings, filter by location, house type, rent range and bedrooms, view property details, and use a separate management window to add, update or delete listings. The application also provides availability information, dashboard statistics, validation and CSV export. This report documents the project in line with the JNEC assignment: database setup, GUI design, CRUD operations and additional features, supported by screenshots of the running interface. The screenshots show the main application, search/filter state, management window and an example filled form. They do not claim that every possible user input has been tested.

1.Introduction

Finding rental accommodation can take time when property details are scattered across different sources. Tenants need to compare rent, location, room count, facilities and availability, while landlords need a way to maintain accurate listing information. Smart Housing brings this information into a single desktop interface and stores records in a local database.

Problem Statement and Objectives

Problem statement: Rental seekers may find it difficult to compare properties consistently, and manual listing updates can lead to incomplete or outdated information.

Objectives:

- Create a searchable desktop application for rental houses, apartments and rooms.
- Display property details and availability in a clear interface.
- Support adding, viewing, updating and deleting listing records.
- Store listing data persistently using SQLite.
- Provide useful additional features such as search/filtering, validation, statistics and CSV export.

1. GUI Design and User Interface

The main window is titled “SMART HOUSING - Rental House Finder”. It uses a dark-blue header, a search/filter area, dashboard summary cards, a Treeview table of available rentals, and a CSV export control. The search inputs include location, house type, minimum rent, maximum rent and bedroom count. The “Manage Listings” button opens a separate window for maintaining records.

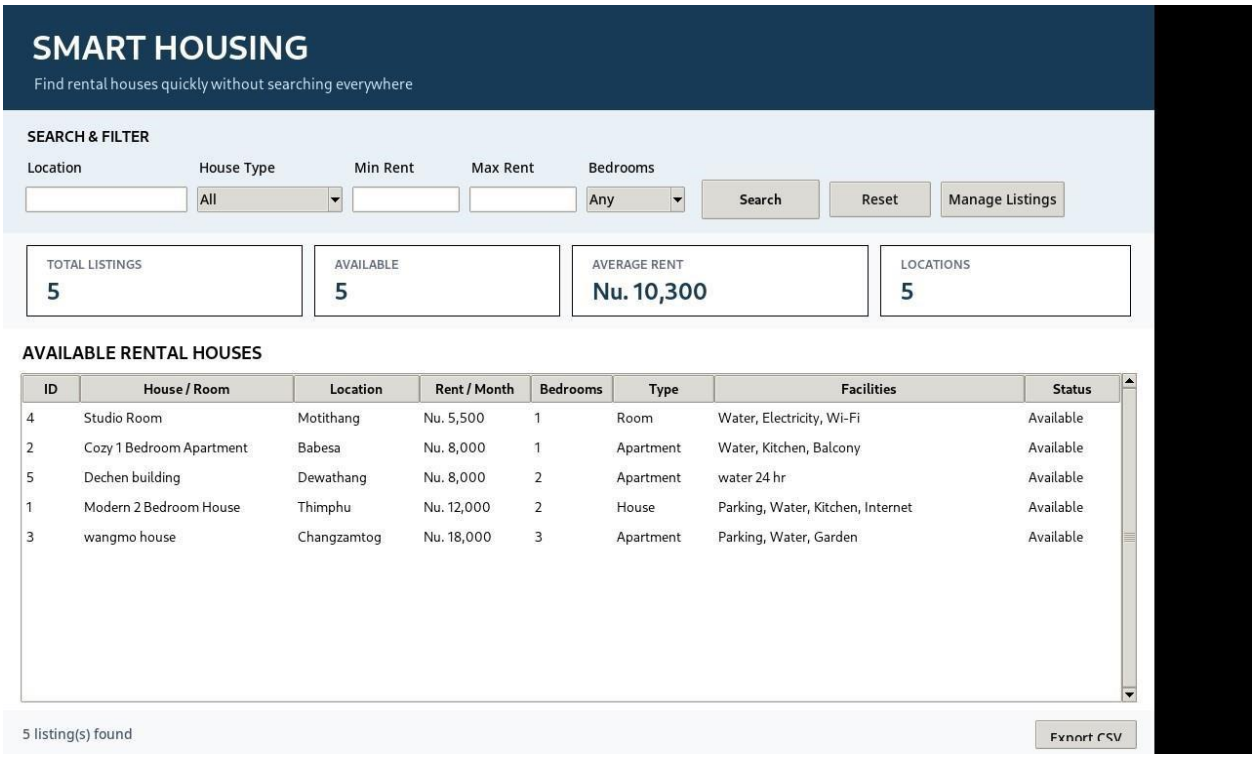


Figure 1 The table shows five records from the supplied database.

1.1 Search and filtering

The user can narrow the visible listings using the location field, house-type dropdown, rent range and bedroom selection. The screenshot below shows the interface with “Babesa” entered as the location filter. The application refreshes the list to show matching records.

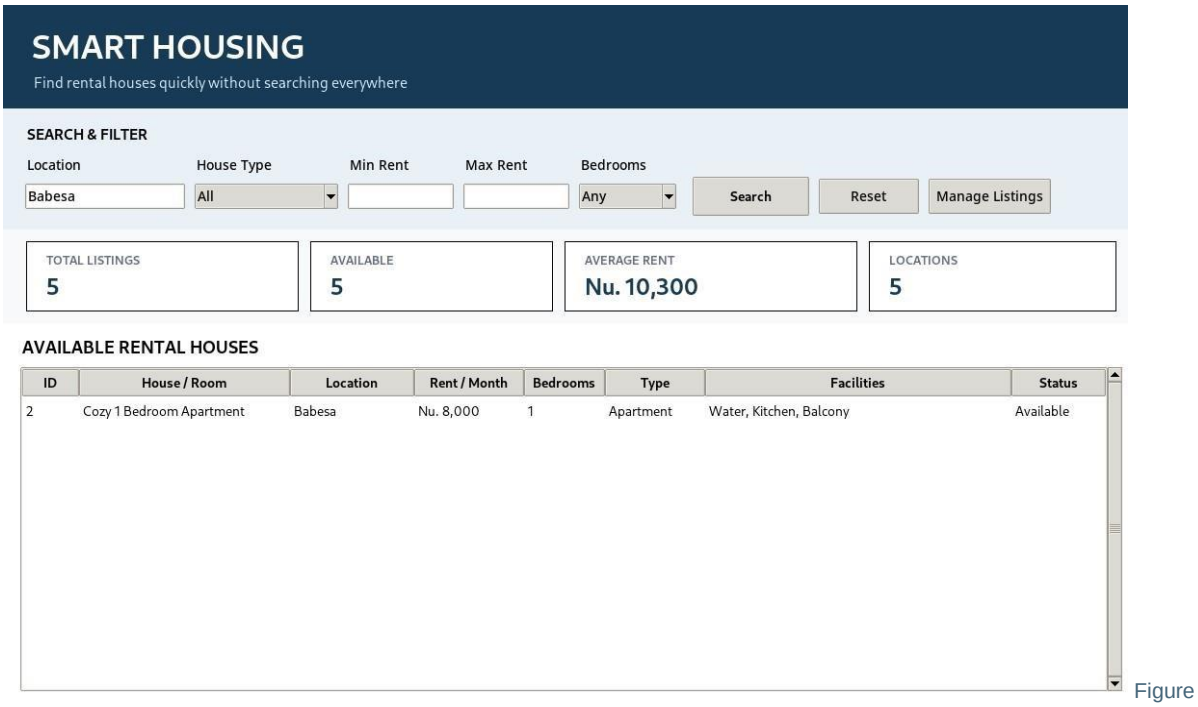


Figure 2. Search/filter interface with a location criterion entered.

2. Database Setup and Design

The application connects to the local SQLite file `smart_housing.db`. The `setup_database()` function creates the `houses` table if it does not already exist. If the table is empty, the application inserts sample records. The supplied database currently contains five rental listings.

The database uses a primary key and constraints to protect basic data integrity. SQL statements use placeholders for inserted and updated values. Foreign-key enforcement is enabled in the connection, although the current schema does not define a foreign-key relationship because the project uses one main table.

	house_id	title	location	rent	bedrooms	house_type	facilities	description	contact	available	last_upd
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Modern 2 Bedroom House	Thimphu	12000.0	2	House	Parking, Water, Kitchen, Internet	Comfortable two-bedroom house …	17XXXXXX	1	2026-09-2
2	2	Cozy 1 Bedroom Apartment	Babesa	8000.0	1	Apartment	Water, Kitchen, Balcony	Affordable apartment close to shops…	17XXXXXX	1	2026-09-2
3	3	wangmo house	Changzamtog	18000.0	3	Apartment	Parking, Water, Garden	Spacious rental house with a small …	17XXXXXX	1	2026-09-2
4	4	Studio Room	Motithang	5500.0	1	Room	Water, Electricity, Wi-Fi	Simple studio room for a single …	17XXXXXX	1	2026-09-2
5	5	Dechen building	Dewathang	8000.0	2	Apartment	water 24 hr	newly build	17843536	1	2026-09-2

3. CRUD Operations

The application includes a separate “Manage Rental Listings” window. The form contains fields for title, location, rent, bedrooms, house type, facilities, contact and description, plus an availability option. The management table lists saved records and provides controls for record maintenance.

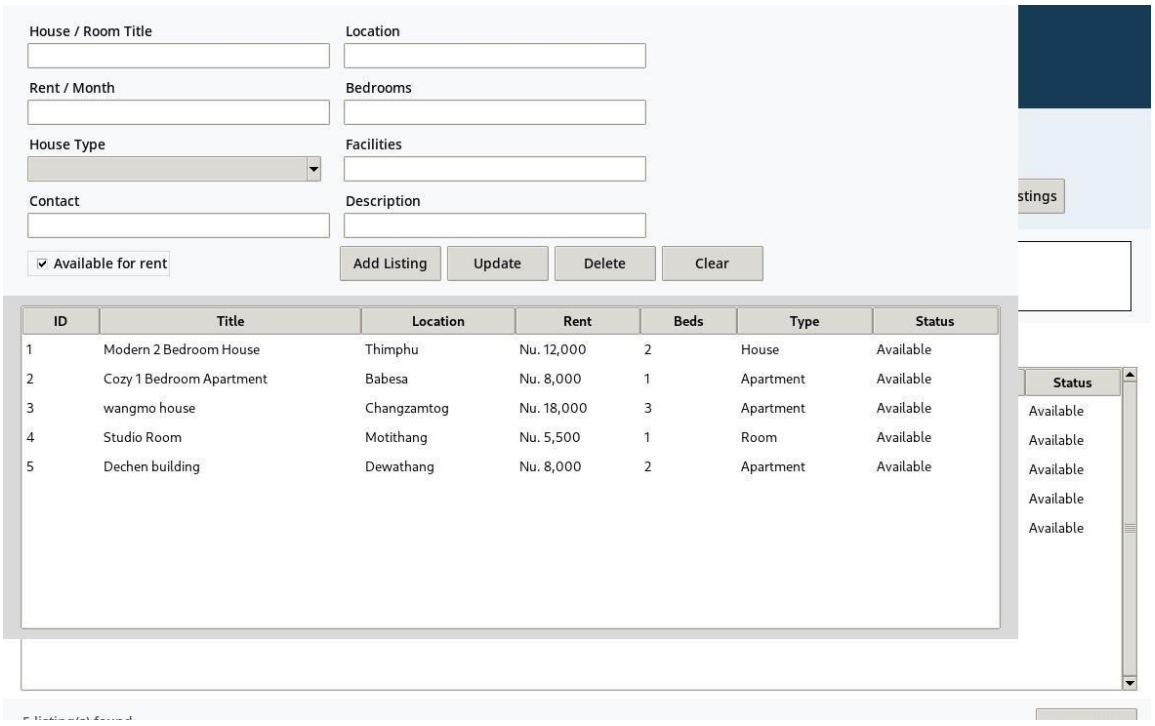


Figure 3. Management window showing the listing form and existing database records.

3.1 Create (Add)

The user enters property details in the form and chooses Add. The `add()` method calls `get_form_data()` to validate the input, inserts the new row into SQLite, refreshes the management table and main listing view, and shows a success message. The filled form below illustrates the kind of information entered before saving; this example entry was not saved to the database.

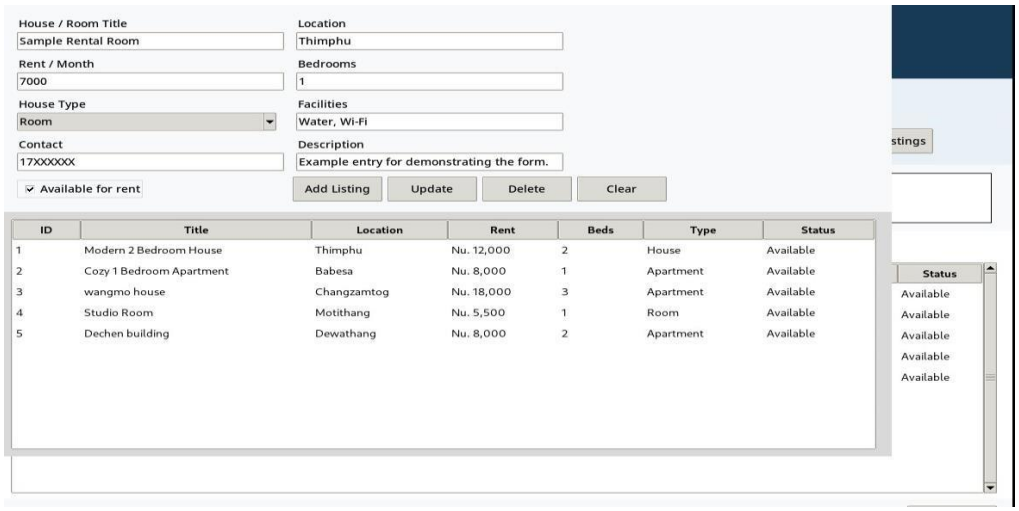


Figure 4. Example completed Add form. The sample form values are illustrative and were not committed to the database.

3.2 Read (View)

The main window reads saved properties from SQLite and displays them in a Treeview table. Selecting a row and using the details function displays further information about that property. The management window also reloads records through its `refresh()` method.

ID	House / Room	Location	Rent / Month	Bedrooms	Type	Facilities	Status
4	Studio Room	Motithang	Nu. 5,500	1	Room	Water, Electricity, Wi-Fi	Available
2	Cozy 1 Bedroom Apartment	Babesa	Nu. 8,000	1	Apartment	Water, Kitchen, Balcony	Available
5	Dechen building	Dewathang	Nu. 8,000	2	Apartment	water 24 hr	Available
1	Modern 2 Bedroom House	Thimphu	Nu. 12,000	2	House	Parking, Water, Kitchen, Internet	Available
3	wangmo house	Changzamtog	Nu. 18,000	3	Apartment	Parking, Water, Garden	Available

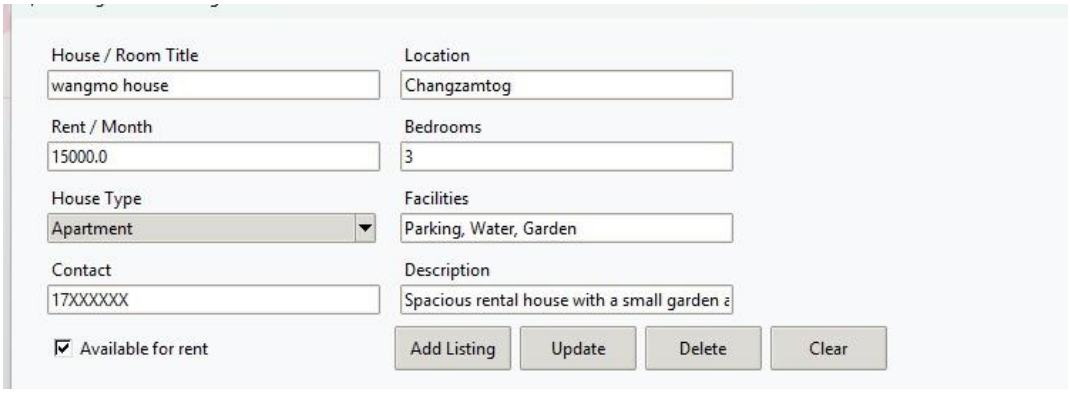
3.3 Update

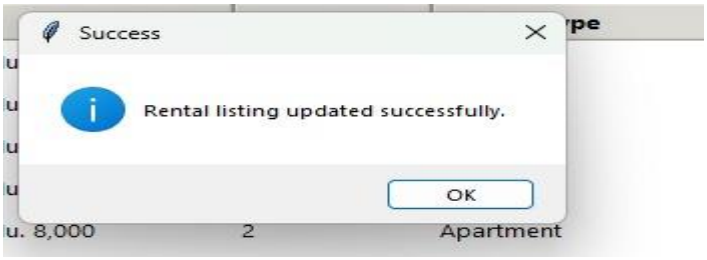
To update a listing, the user selects a row in the management table, loads or enters the revised values, and chooses Update. The `update()` method checks that a record is selected, validates the form data, updates the chosen row using its house ID, and refreshes both views.

Before update:



After update:

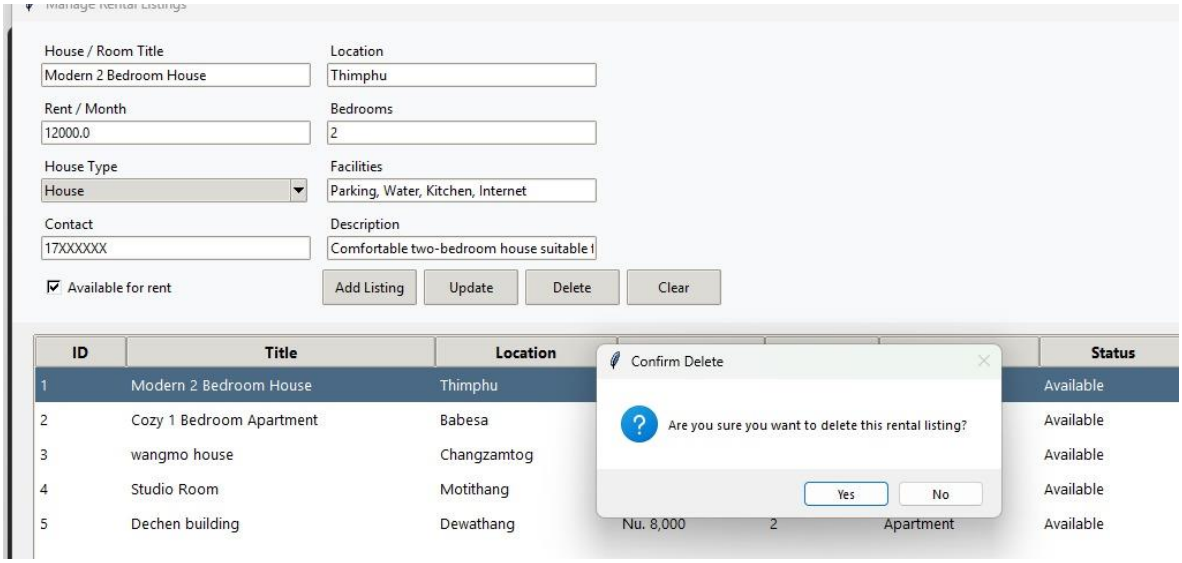


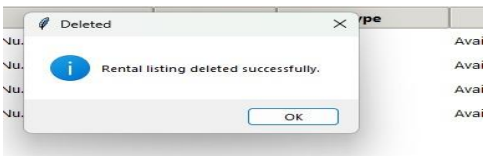


3.4 Delete

To delete a listing, the user selects a row and chooses Delete. The application asks for confirmation before removing the record. If the user cancels, the record is retained; if confirmed, the selected row is deleted and the list is refreshed.

Note on evidence: Figures 3–4 show the actual management interface and form. They are not screenshots of each individual database mutation. The Add, Update and Delete steps above describe the corresponding methods in the supplied source code; execute these actions in the application and capture them separately if your lecturer requires evidence of each operation.

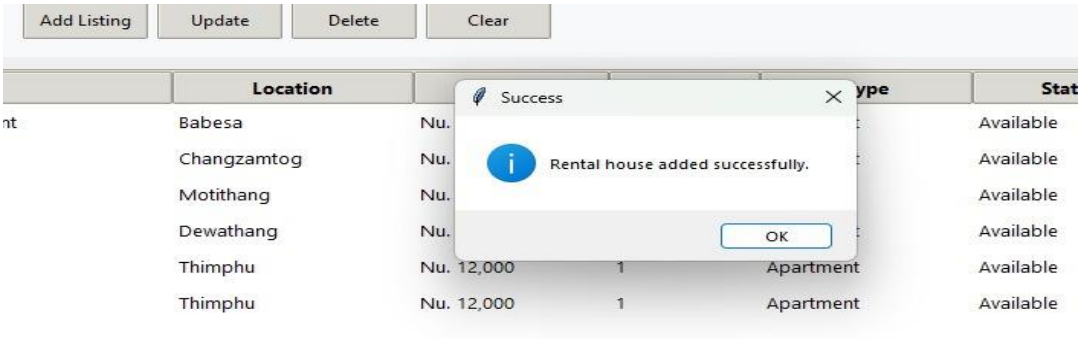




4. Additional Features

Feature	How it works / benefit
Search and filtering	Filters listings by location, house type, minimum/maximum rent and bedrooms.
Data validation	Checks form fields and numeric values before a listing is stored.
CSV export	Exports listing information to a CSV file for use in spreadsheet software.
Dashboard statistics	Displays total listings, available listings, average rent and number of locations.
Multi-window navigation	Separates rental browsing from the listing-management interface.
Availability status	Stores and displays whether a property is available or rented.

These features exceed the assignment minimum of two additional features. Authentication and role-based access are not implemented in the supplied version.



5. Code Quality and Organization

The project organizes its logic into functions and classes. Database connection and initialization are handled by `connect_db()` and `setup_database()`. The `SmartHousingApp` class manages the main window, search, table refresh, statistics, property details and CSV export. The `ManageListings` class handles listing management and CRUD actions. Comments divide the source code into logical sections. SQL placeholders are used for data values.

6. Testing and Evaluation

The supplied source and database were inspected, and the application was launched to capture the interface screenshots included in this report. The existing database was confirmed to contain five listing records. The table below distinguishes these observations from actions that still need manual verification on the student's computer.

Test	Expected result	Evidence/status
Launch application	Main window opens and records are shown.	Observed for screenshot capture.
Database initialization	houses table exists; sample records are inserted if empty.	Source/schema inspected; current DB has 5 rows.

Search by location	Results narrow to matching location.	Search state captured in Figure 2; verify interactively.
View property details	Details for selected property are displayed.	Implemented in source; manual test recommended.
Add listing	Valid form creates a new row and refreshes views.	Implemented in source; manual test recommended.
Update listing	Selected row reflects edited values.	Implemented in source; manual test recommended.
Delete listing	Confirmation is shown and selected row is removed only if confirmed.	Implemented in source; manual test recommended.
Validation	Invalid or missing data is rejected with feedback.	Validation method inspected; test boundary cases.
CSV export	CSV file is created with listing data.	Implemented in source; manual test recommended.

Evaluation: the application addresses the assignment's key design requirements through a working Tkinter interface, SQLite storage, CRUD methods and several additional features. To complete the evidence for submission, run each CRUD action and add screenshots showing the result messages and updated table. Test outcomes should be recorded only after the actions have been performed.

7. Limitations and Future Enhancements

- The app is a local desktop prototype and does not provide shared online access.
- Login/authentication and role-based permissions are not implemented.
- Listing contact details and availability depend on the accuracy of user-entered data.
- Future improvements could include property photos, sorting by table column, secure user accounts, backups, audit history and a web/mobile version.

8. Conclusion

Smart Housing is a Python desktop rental finder that uses Tkinter/ttk for the interface and SQLite for data storage. It provides listing search and filtering, a table view, property details, listing management, availability status, dashboard statistics, validation and CSV export. This report documents the database design, interface, CRUD methods and additional features using screenshots of the actual running application. Final submission should include the Python file, SQLite database, README and this report, with any additional screenshots required by the lecturer.